

Universidad de la Sierra Sur

Integración del evento de Amazon S3 con AWS Lambda, logs, métricas y alarma para la Gaceta Universitaria «El Unsito»

MANUAL DE IMPLEMENTACIÓN

PROYECTO INTEGRADOR · EQUIPO 3
TECNOLOGÍAS DE LA INFORMACIÓN I

Presentan:

Daniel González Ruiz
Mayra Castellanos Pacheco
Gael Eduardo García Pinacho
Elisa Sánchez Martínez

Docente:

M.C.C. Eliezer Alcazar Silva

Miahuatlán de Porfirio Díaz, Oaxaca.
Octubre de 2026

Contenido

1. Antes de empezar 2
2. Preparar la función de prefirmadas 2
3. Crear la función procesadora 2
4. Conectar S3 con la función 3
5. Actualizar la consulta y el sitio 3
6. Configurar los reintentos 4
7. Configurar la retención de logs 4
8. Crear la alarma 4
9. Asignar un rol de mínimo privilegio 5
10. Verificación 6
11. Consumo y limpieza 10

1. Antes de empezar

Este manual describe los pasos que siguió el Equipo 3 en la consola de AWS para que la carga de una imagen en S3 active la función que guarda la noticia, y para vigilar esa función con logs, una métrica y una alarma. La arquitectura y las decisiones de diseño se explican en la ficha técnica.

- Región: us-east-2 (Ohio). Todos los recursos deben estar en la misma región.
- Recursos existentes: el bucket `mis-noticias-daniel-imagenes-906-2026`, la tabla `noticias` y las funciones `cors`, `agregarNoticia` y `obtenerNoticias`.
- Código: la carpeta `codigo/` de los entregables contiene las funciones y la política IAM.

> En las capturas, el recuadro rojo señala el dato que se configura o se comprueba en cada paso.

2. Preparar la función de prefirmadas

Antes de cambiarla, se respaldó el código de la función `cors`. Después se reemplazó por la versión nueva, que recibe los datos de la noticia, los guarda en `pendientes/ID.json` y devuelve la URL para subir la imagen a `imagenes/ID/foto.ext`. La versión nueva conserva el modo anterior para no afectar a quien tenga la página antigua en caché.

3. Crear la función procesadora

1. En **Lambda › Crear función**, crear `procesar-imagen-noticia` con Node.js 24.x.
2. Pegar el código de `procesar-imagen.mjs` y pulsar **Deploy**.
3. En **Configuración general**, subir el tiempo de espera a 30 segundos.
4. En **Variables de entorno**, agregar `TABLA` y `BUCKET` (figura 1).

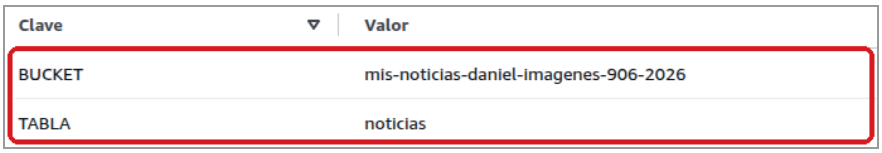


Figura 1. Variables de entorno de la procesadora. `TABLA` permite simular un fallo sin tocar la tabla real.

4. Conectar S3 con la función

1. En la procesadora, pulsar **Agregar desencadenador › S3**.
2. Elegir el bucket de imágenes, el tipo **Todos los eventos de creación de objetos** y el prefijo `imagenes/`.
3. Aceptar el aviso de invocación recursiva: no hay ciclo, porque la función no escribe en S3.



Figura 2. Desencadenador creado: bucket, prefijo `imagenes/` y evento `s3:ObjectCreated:*`.

Lambda crea por sí sola el permiso para que S3 invoque la función. La regla también aparece en las propiedades del bucket (figura 3).



Figura 3. Notificación de eventos del bucket con destino en la procesadora.

5. Actualizar la consulta y el sitio

Se reemplazó el código de `obtenerNoticias` para que también responda la consulta `?id=` con el estado `pendiente` o `publicada`. Después se subió el nuevo `main.js` al bucket del sitio y se cambió su número de versión en `index.html`; sin ese cambio, el navegador seguía usando el archivo anterior guardado en caché.

6. Configurar los reintentos

En **Configuración › Invocación asíncrona** se fijaron dos reintentos y una antigüedad máxima del evento de una hora.

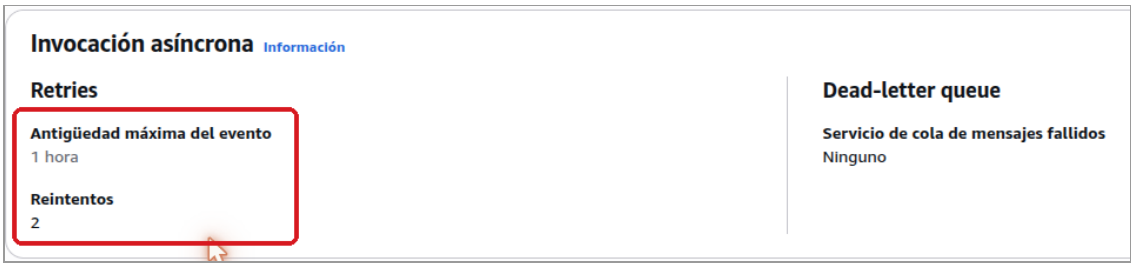


Figura 4. Reintentos de la invocación asíncrona.

7. Configurar la retención de logs

En **CloudWatch › Grupos de registros**, se abrió `/aws/lambda/procesar-imagen-noticia` y en **Acciones › Editar configuración de retención** se eligieron tres días.

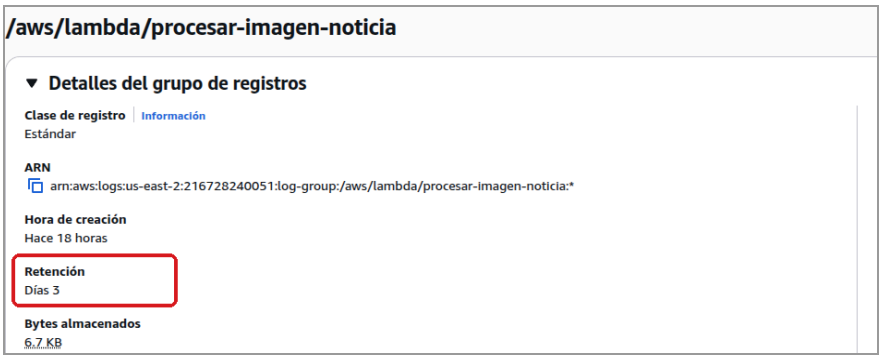


Figura 5. Grupo de logs de la procesadora con retención de tres días.

8. Crear la alarma

1. En **CloudWatch › Alarmas › Crear alarma**, elegir la métrica `Errors` de la función `procesar-imagen-noticia`.
2. Configurar la condición: suma mayor o igual que 1 en un periodo de 1 minuto, con 1 de 1 puntos de datos.
3. Tratar los datos faltantes como correctos, no agregar acciones y nombrarla `gaceta-procesadora-errores`.

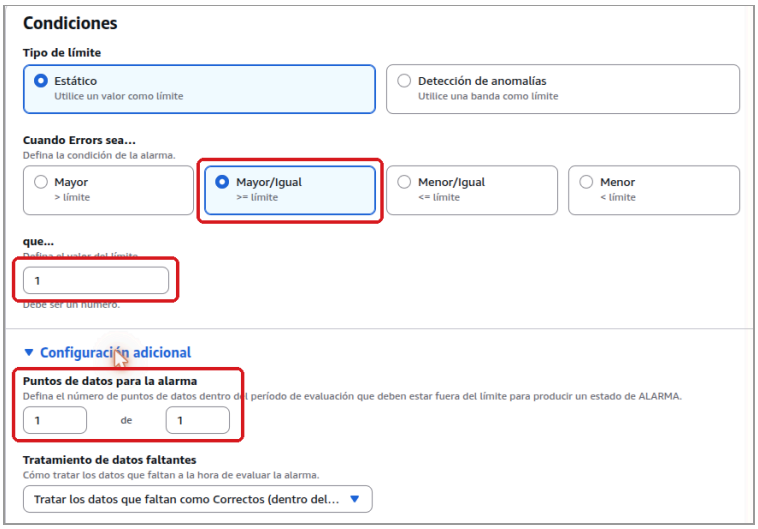


Figura 6. Métrica y condiciones de la alarma.

9. Asignar un rol de mínimo privilegio

1. En **IAM › Roles › Crear rol**, elegir **Lambda** como servicio de confianza, sin políticas administradas.
2. Nombrarlo `procesar-imagen-noticia-rol` y agregar las etiquetas del proyecto.
3. En **Agregar permisos › Crear política insertada › JSON**, pegar `procesadora-politica.json`. El validador debe mostrar cero problemas (figura 7).
4. En la procesadora, en **Configuración general › Editar**, cambiar el rol de ejecución por el nuevo.



Figura 7. Política insertada: logs propios, lectura de `pendientes/` y escritura en la tabla.



Figura 8. Rol creado con la política `ProcesadoraMinimoPrivilegio` como único permiso.

10. Verificación

Con todo configurado, el 27 de octubre de 2026 fuimos comprobando el flujo paso a paso: primero en condiciones normales y después provocando fallos a propósito. Usamos el sitio publicado y la pestaña **Probar** de la procesadora, con los eventos de ejemplo de la carpeta `pruebas/` del repositorio.

10.1. Publicación de una noticia

Lo primero fue publicar una noticia con imagen desde el sitio, como lo haría cualquier usuario. En la red del navegador ya no apareció ninguna llamada a `agregarNoticia`, y en los logs se ven las tres etapas del procesamiento.



Figura 9. Logs de la primera publicación: `inicio`, `validado` y `guardado`, en 1.56 segundos.

Al revisar la tabla, la noticia estaba ahí, y al recargar el sitio aparecía con su imagen.



Figura 10. Noticia guardada en DynamoDB. El campo `origen = s3-evento` indica que la guardó la procesadora.

10.2. Una imagen sin datos

Después quisimos ver qué pasa si llega una imagen cuyos datos no existen. Enviamos ese evento y la función registró `validacion_fallida` con el motivo y terminó sin error, porque reintentar no lo corregiría. No se creó ninguna noticia incompleta.

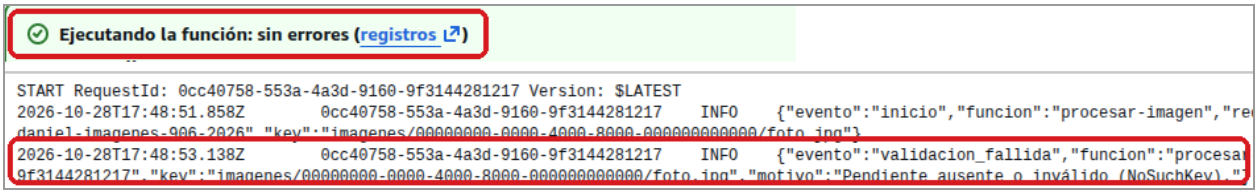


Figura 11. Ejecución sin errores y log con el motivo «Pendiente ausente o inválido (NoSuchKey)».

10.3. Una caída de la base de datos

El caso más importante era un fallo al guardar. Para simularlo sin dañar la tabla del grupo, cambiamos la variable `TABLA` por una tabla inexistente y publicamos una noticia. La función falló tres veces con el mismo identificador de solicitud: el intento original y los dos reintentos. Al restaurar `TABLA = noticias` y reprocesar el evento, la noticia se guardó con el mismo id.

Marca temporal	Mensaje
2026-10-27T23:45:50.809Z	2026-10-27T23:45:50.809Z 5fc608fe-f516-4d9f-90c2-b12448fb6cd1 INFO {"evento":"inicio","funcion":"procesar-im
2026-10-27T23:45:51.931Z	2026-10-27T23:45:51.931Z 5fc608fe-f516-4d9f-90c2-b12448fb6cd1 INFO {"evento":"validado","funcion":"procesar-
2026-10-27T23:45:52.268Z	2026-10-27T23:45:52.268Z 5fc608fe-f516-4d9f-90c2-b12448fb6cd1 ERROR {"evento":"fallo","funcion":"procesar-im
2026-10-27T23:45:52.288Z	2026-10-27T23:45:52.288Z 5fc608fe-f516-4d9f-90c2-b12448fb6cd1 ERROR Invoke Error {"errorType":"Error","erro
2026-10-27T23:46:54.469Z	2026-10-27T23:46:54.469Z 5fc608fe-f516-4d9f-90c2-b12448fb6cd1 INFO {"evento":"inicio","funcion":"procesar-im
2026-10-27T23:46:54.615Z	2026-10-27T23:46:54.615Z 5fc608fe-f516-4d9f-90c2-b12448fb6cd1 INFO {"evento":"validado","funcion":"procesar-
2026-10-27T23:46:54.710Z	2026-10-27T23:46:54.710Z 5fc608fe-f516-4d9f-90c2-b12448fb6cd1 ERROR {"evento":"fallo","funcion":"procesar-im
2026-10-27T23:46:54.729Z	2026-10-27T23:46:54.729Z 5fc608fe-f516-4d9f-90c2-b12448fb6cd1 ERROR Invoke Error {"errorType":"Error","erro
2026-10-27T23:48:48.892Z	2026-10-27T23:48:48.892Z 5fc608fe-f516-4d9f-90c2-b12448fb6cd1 INFO {"evento":"inicio","funcion":"procesar-im
2026-10-27T23:48:49.520Z	2026-10-27T23:48:49.520Z 5fc608fe-f516-4d9f-90c2-b12448fb6cd1 INFO {"evento":"validado","funcion":"procesar-
2026-10-27T23:48:49.651Z	2026-10-27T23:48:49.651Z 5fc608fe-f516-4d9f-90c2-b12448fb6cd1 ERROR {"evento":"fallo","funcion":"procesar-im
2026-10-27T23:48:49.652Z	2026-10-27T23:48:49.652Z 5fc608fe-f516-4d9f-90c2-b12448fb6cd1 ERROR Invoke Error {"errorType":"Error","erro
2026-10-27T23:50:25.827Z	2026-10-27T23:50:25.827Z 5921aa4e-187a-4326-918c-4c37548924ca INFO {"evento":"inicio","funcion":"procesar-im
2026-10-27T23:50:27.048Z	2026-10-27T23:50:27.048Z 5921aa4e-187a-4326-918c-4c37548924ca INFO {"evento":"validado","funcion":"procesar-
2026-10-27T23:50:27.308Z	2026-10-27T23:50:27.308Z 5921aa4e-187a-4326-918c-4c37548924ca INFO {"evento":"guardado","funcion":"procesar-

Figura 12. Tres errores `ResourceNotFoundException` a las 23:45, 23:46 y 23:48 UTC.

10.4. La métrica y la alarma

Ese mismo fallo nos sirvió para comprobar la vigilancia. La métrica `Errors` registró el pico y la alarma se encendió a las 23:46:42; al corregir la tabla volvió a la normalidad a las 23:51:42.

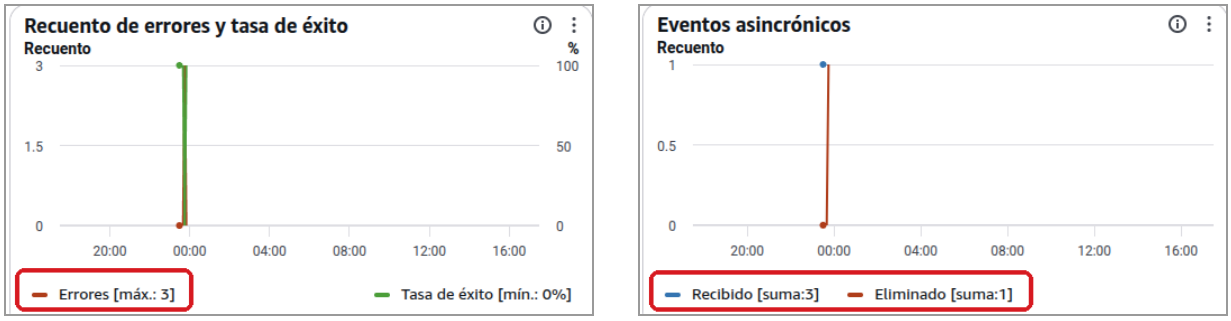


Figura 13. Pico de errores (izquierda) y eventos asíncronos recibidos y eliminados (derecha).

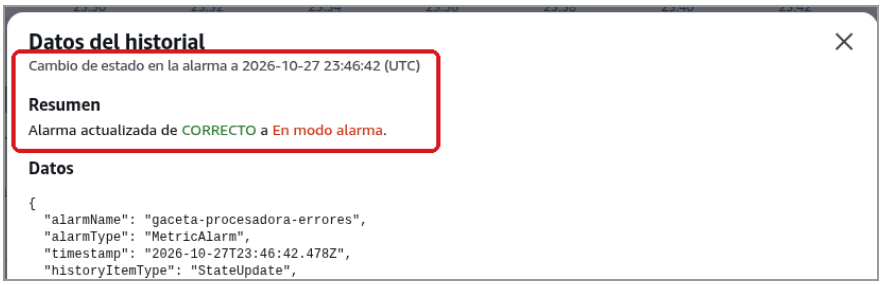


Figura 14. La alarma cambia de «Correcto» a «En modo alarma».



Figura 15. La franja roja marca el tiempo en alarma; después vuelve a «Correcto».

10.5. Un evento repetido

S3 puede entregar el mismo evento más de una vez, así que volvimos a enviar el de la primera publicación. La función registró `duplicado` y la tabla conservó un solo registro.

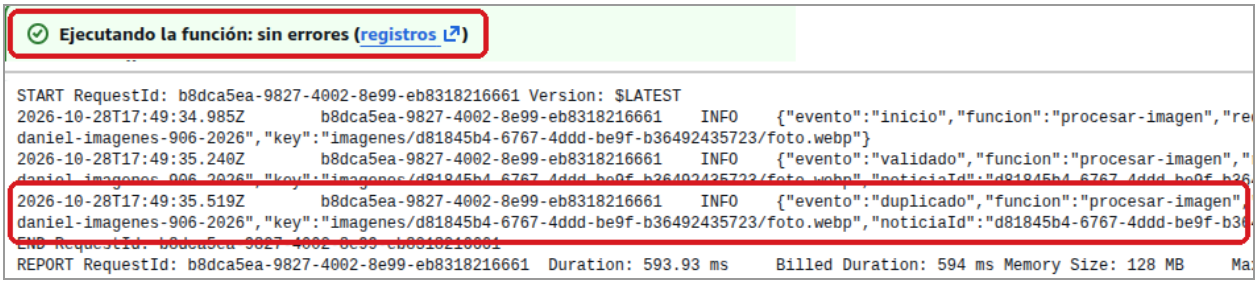


Figura 16. Log `duplicado` para la misma noticia.

10.6. Un procesamiento lento

Por último, revisamos qué veía el usuario mientras la base de datos estaba caída. La web consultó el estado diez veces y avisó que la noticia seguía en procesamiento, sin volver a enviarla. Tras la recuperación apareció una sola vez.

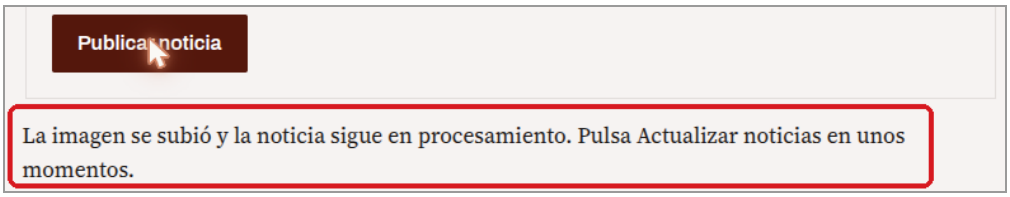


Figura 17. Mensaje del sitio mientras la noticia sigue pendiente.

11. Consumo y limpieza

El saldo fue de 120.00 USD antes y después de la verificación, sin gasto en el mes.



Figura 18. Créditos restantes y gasto del mes al terminar.

Al terminar el curso, eliminar los recursos en este orden:

1. El desencadenador de S3 de la procesadora.
2. La función `procesar-imagen-noticia` y su rol.
3. La alarma `gaceta-procesadora-errores`.
4. El grupo de logs `/aws/lambda/procesar-imagen-noticia`.
5. Los objetos de `pendientes/`, las imágenes de prueba y las noticias de prueba de la tabla.

Si el sitio sigue en uso, antes se restaura en `cors` el código original (`prefirmadas-original.mjs`).